T02.03 CI/CD en GitLab

Miguel Tubío Vidal

Exercicio 1: Probas automatizadas mediante CI/CD

1.1 Creación do proxecto e do código

Creouse en GitLab.com o proxecto `mtvt0204` co ficheiro `README.md`. Dende **Código** → **Web IDE** (un VS Code online) creáronse os ficheiros da aplicación e fíxose un commit; dende o Web IDE o push realízase automaticamente.

`dni_validator.py` (función aínda sen implantar, sempre devolve `True`):

```
import re

def validar_dni(dni):
    return True
```

`test_dni_validator.py` (test unitarios con `unittest`):

```
import unittest
from dni_validator import validar_dni

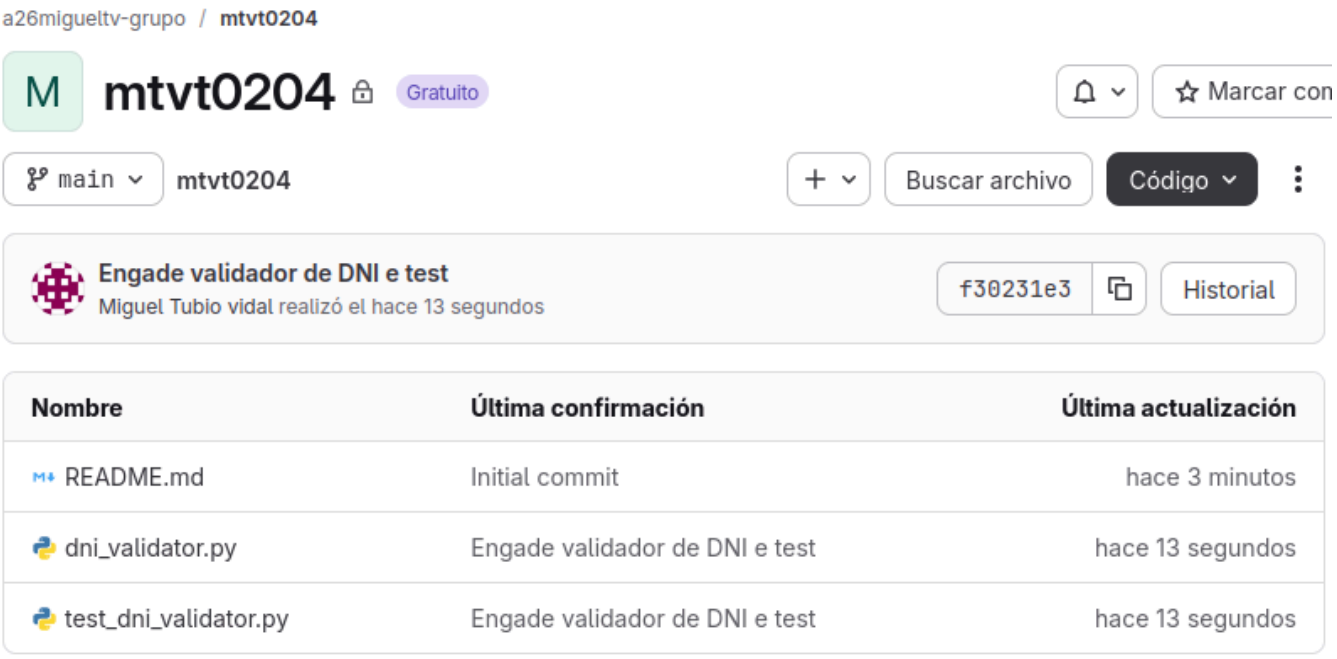
class TestDniValidator(unittest.TestCase):

    def test_dni_correctos(self):
        self.assertTrue(validar_dni("12345678Z"))  # Exemplo válido
        self.assertTrue(validar_dni("00000000T"))
        self.assertTrue(validar_dni("98765432M"))
        self.assertTrue(validar_dni("99999999R"))

    def test_dni_incorrectos(self):
        self.assertFalse(validar_dni("12345678A"))  # Letra incorrecta
        self.assertFalse(validar_dni("1234567Z"))   # Faltan números
        self.assertFalse(validar_dni("123456789Z")) # Sobran números
        self.assertFalse(validar_dni("ABCDEFGHZ"))  # Letras no número
        self.assertFalse(validar_dni("12345678"))   # Sen letra
        self.assertFalse(validar_dni(""))           # Baleiro

if __name__ == '__main__':
    unittest.main()
```

Captura 1 – Interface web do GitLab co contido do proxecto



1.2 Instalación de GitLab Runner na máquina runnermtv

Conectouse por SSH dende Visual Studio Code á máquina `runnermtv` (`192.168.56.104`) co usuario `dadmin`:

```
ssh dadmin@192.168.56.104
```

Docker non estaba instalado porque a condición do Vagrantfile (`m[:hostname] == "runner"`) non coincidía co nome da máquina (`runnermtv`). Corrixiuse por `m[:hostname].start_with?("runner")` e executouse `vagrant provision runnermtv` dende o equipo anfitrión.

Comprobouse que Docker está instalado, xa que o executor `docker` o necesita:

```
docker --version
```

Instalouse GitLab Runner dende o repositorio oficial de GitLab:

```
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-
runner/script.deb.sh" -o script.deb.sh
sudo bash script.deb.sh
sudo apt install -y gitlab-runner
```

Engadiuse o usuario `gitlab-runner` (o usuario co que se executa o servizo) ao grupo `docker`, para que poida crear contedores:

```
sudo usermod -aG docker gitlab-runner
```

1.3 Creación e rexistro do runner

En GitLab: **Ajustes → CI/CD → Runners → Crear ejecutor de proyecto**, coa etiqueta `rundock`. GitLab proporciona un comando de rexistro cun token (`glrt-...`), que se executou en `runnermtv`:

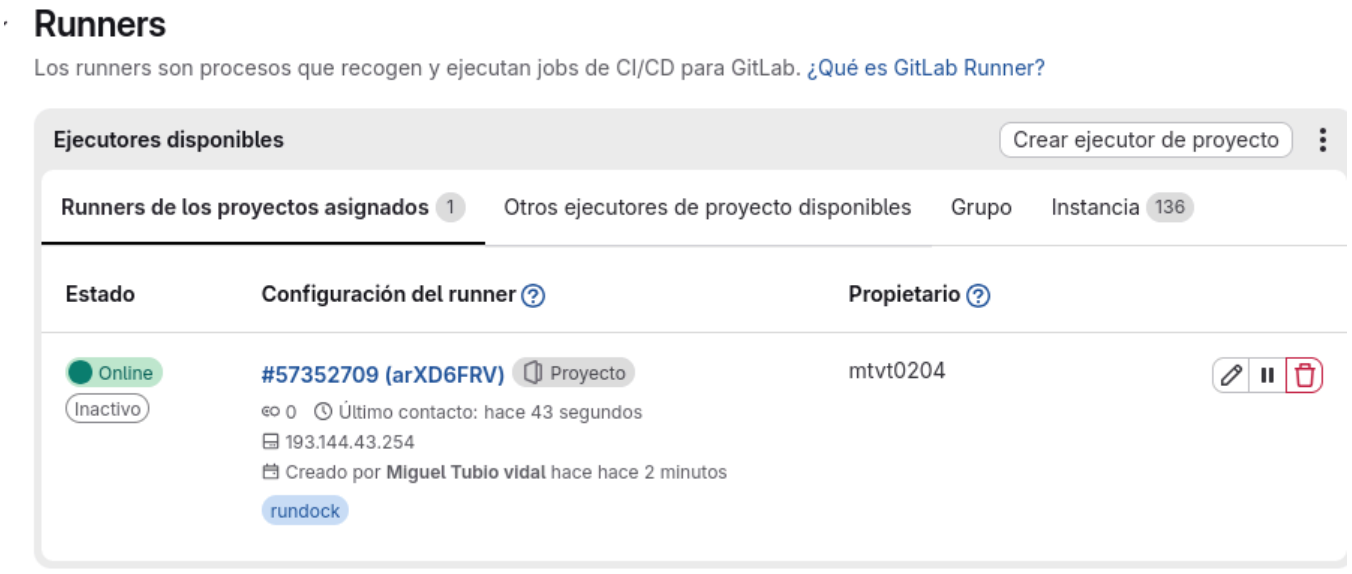
```
sudo gitlab-runner register --url https://gitlab.com --token <TOKEN>
```

Opcións seleccionadas:

Pregunta	Resposta
URL	(por defecto)
Name	`dock-runner-1`
Executor	`docker`
Imaxe por defecto	`debian`

> A imaxe por defecto úsase só nos jobs que non indican a súa propia `image:`.

Captura 2 – Interface web de GitLab co runner creado



1.4 Configuración do pipeline de probas

Creouse o ficheiro `.gitlab-ci.yml` cunha stage `test` e un job `unittest`:

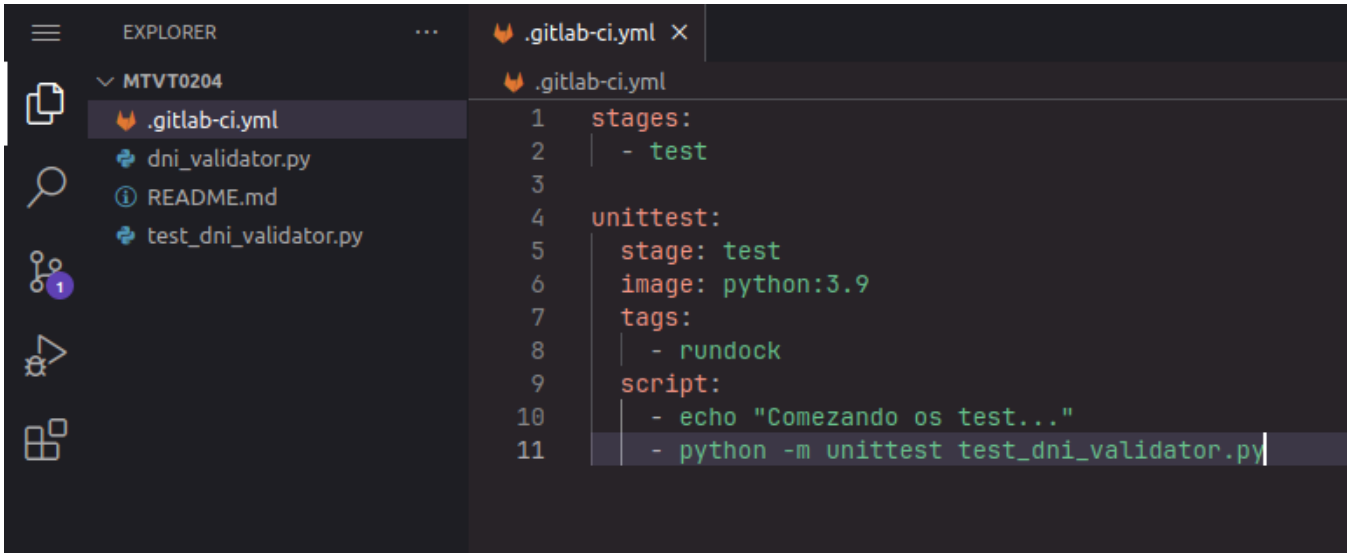
```yaml
stages:
  - test

unittest:
  stage: test
  image: python:3.9
  tags:
    - rundock
  script:
    - echo "Comezando os test..."
    - python -m unittest test_dni_validator.py
```

- `stages`: lista de etapas do pipeline, na orde en que se executan.
- `image`: imaxe Docker do contedor onde corre o job (aquí, Python 3.9).
- `tags`: fai que o job o execute o noso runner `rundock`.
- `script`: comandos que se executan dentro do contedor.

Como a conta non está verificada, desactiváronse os runners de GitLab en **Ajustes → CI/CD → Runners → Instance → Activar ejecutores de instancia en este proyecto** (desactivado).

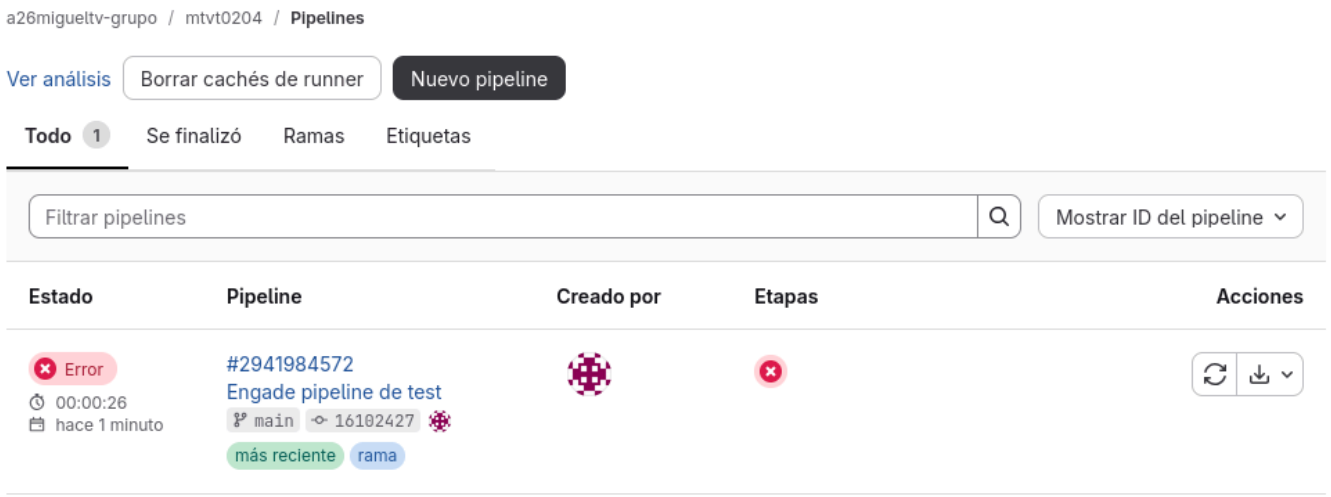
Captura 3 – Contido do ficheiro .gitlab-ci.yml executando o test



1.5 Resultado das probas coa función sen implantar

Tras o push, o pipeline lanzouse automaticamente. Como `validar_dni` sempre devolve `True`, o test `test_dni_incorrectos` falla e o pipeline queda en estado **Failed**.

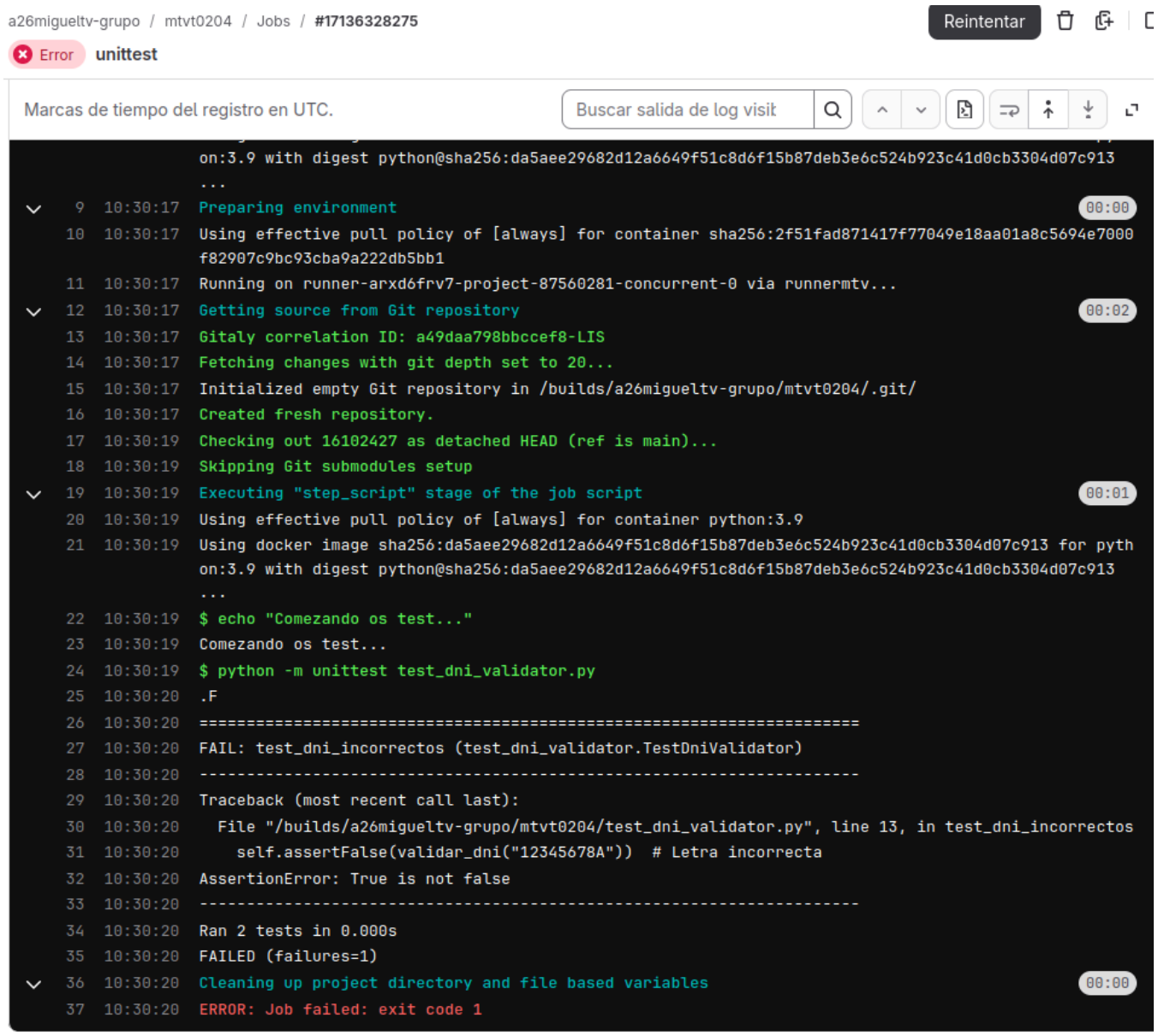
Captura 4 – Build > Pipelines que se executou



Captura 5 – Resumo do Pipeline errado



Captura 6 – Resumo do job errado



1.6 Implantación da función e probas correctas

Modificouse `dni_validator.py` coa implantación real:

```
import re

def validar_dni(dni):
    """
    Valida se un DNI español é correcto en formato e letra.

    Args:
        dni (str): DNI a validar (ex: '12345678Z')

    Returns:
        bool: True se é válido, False se non.
    """
    dni = dni.upper()
    patron = r'^\d{8}[A-Z]$'

    if not re.match(patron, dni):
```

```
        return False

    letras = "TRWAGMYFPDXBNJZSQVHLCKE"
    numero = int(dni[:8])
    letra_correcta = letras[numero % 23]

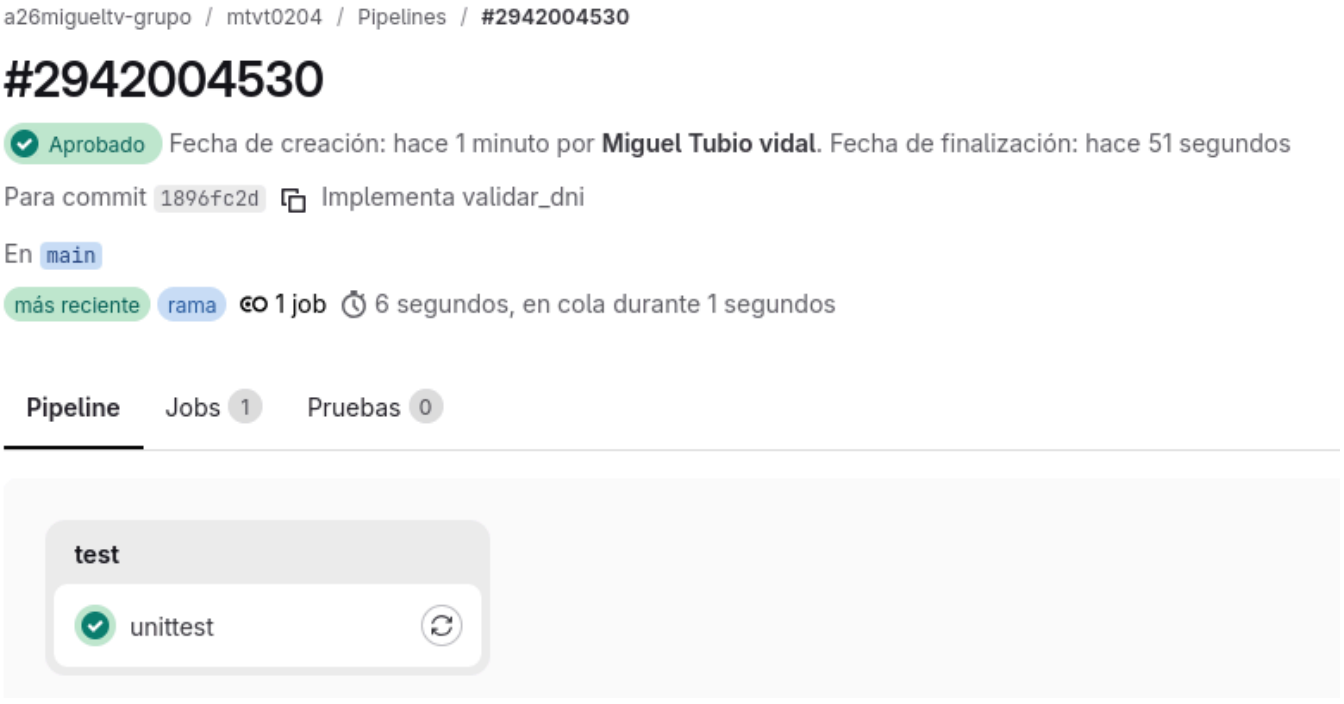
    return dni[-1] == letra_correcta
```

Tras o commit, o novo pipeline remata en estado **Passed**.

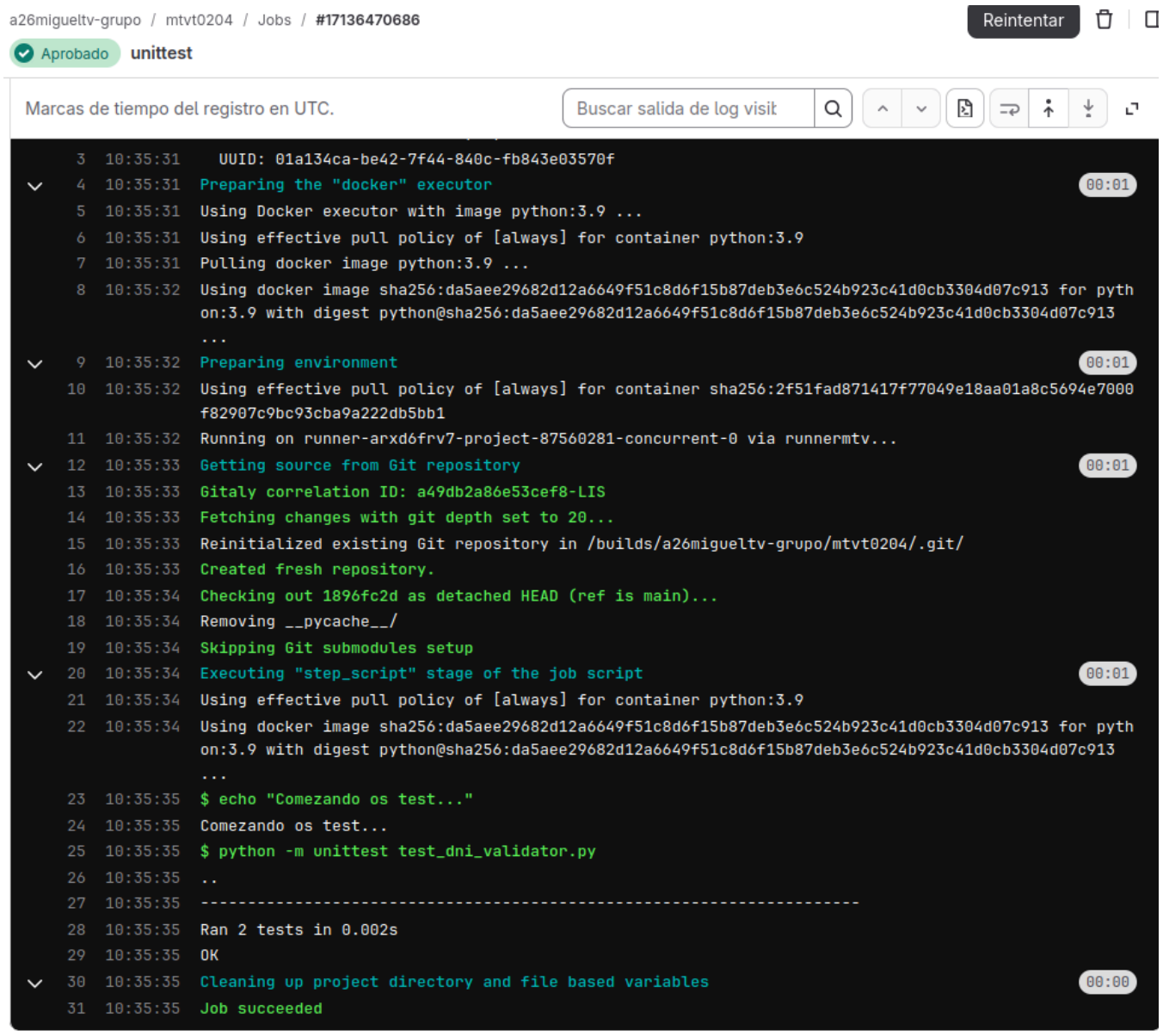
Captura 7 – Build > Pipelines que se executou



Captura 8 – Resumo do Pipeline executado con éxito



Captura 9 – Resumo do job executado con éxito



Exercicio 2: Empaquetado automatizado mediante CI/CD

2.1 Programa principal

Creouse `main.py`, que recibe un DNI por argumento e indica se é válido usando a función anterior:

```
import sys
from dni_validator import validar_dni

def main():
    if len(sys.argv) != 2:
        print("Uso: python main.py <DNI>")
        sys.exit(1)

    dni = sys.argv[1]
    if validar_dni(dni):
        print(f"O DNI '{dni}' é válido.")
    else:
        print(f"O DNI '{dni}' NON é válido.")
```

```
if __name__ == "__main__":
    main()
```

2.2 Nova stage de empaquetado

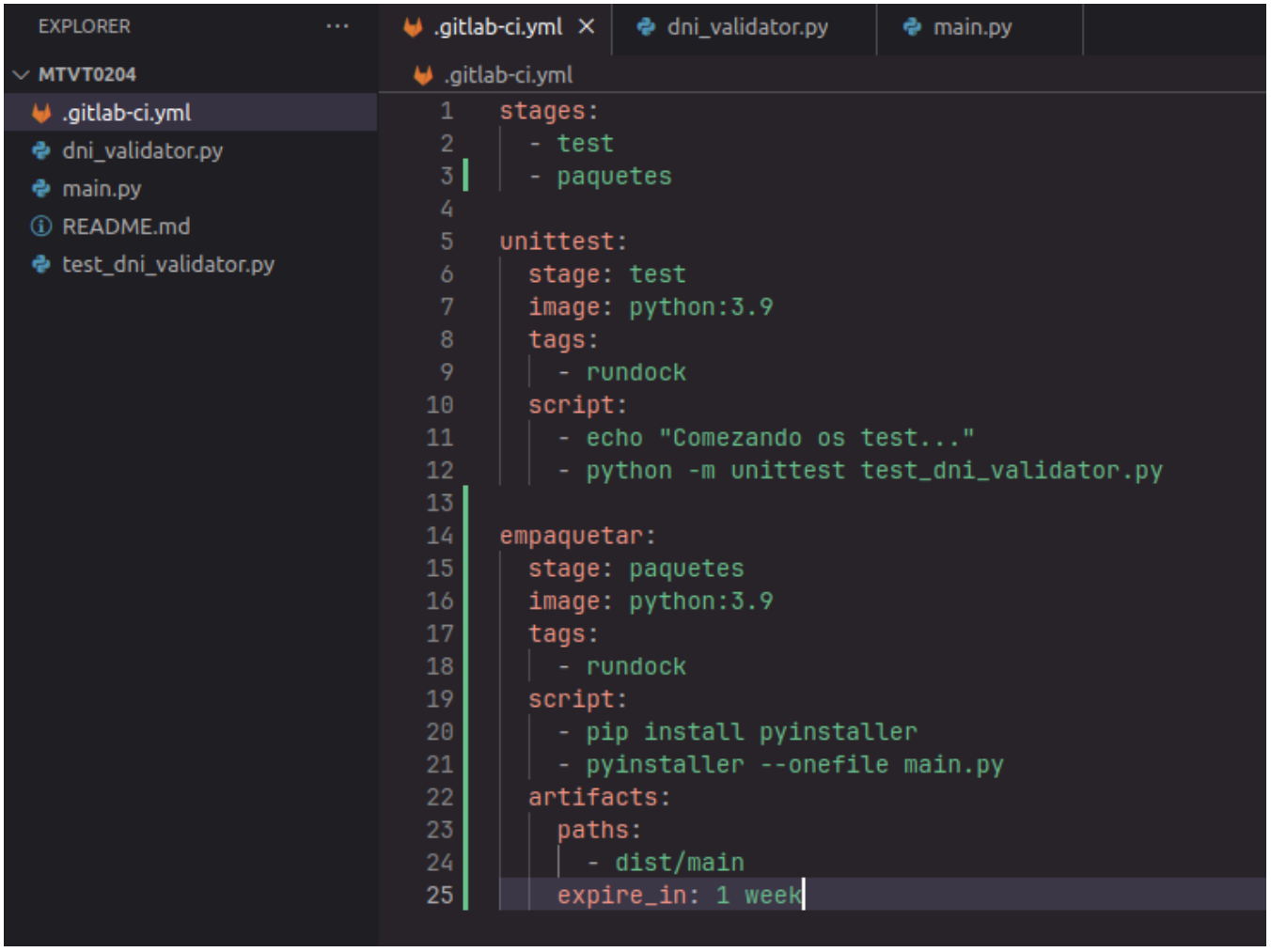
Engadiuse a stage `paquetes` despois de `test` e o job `empaquetar`. Ao estar despois de `test`, **só se executa se as probas pasan**:

```
stages:
  - test
  - paquetes

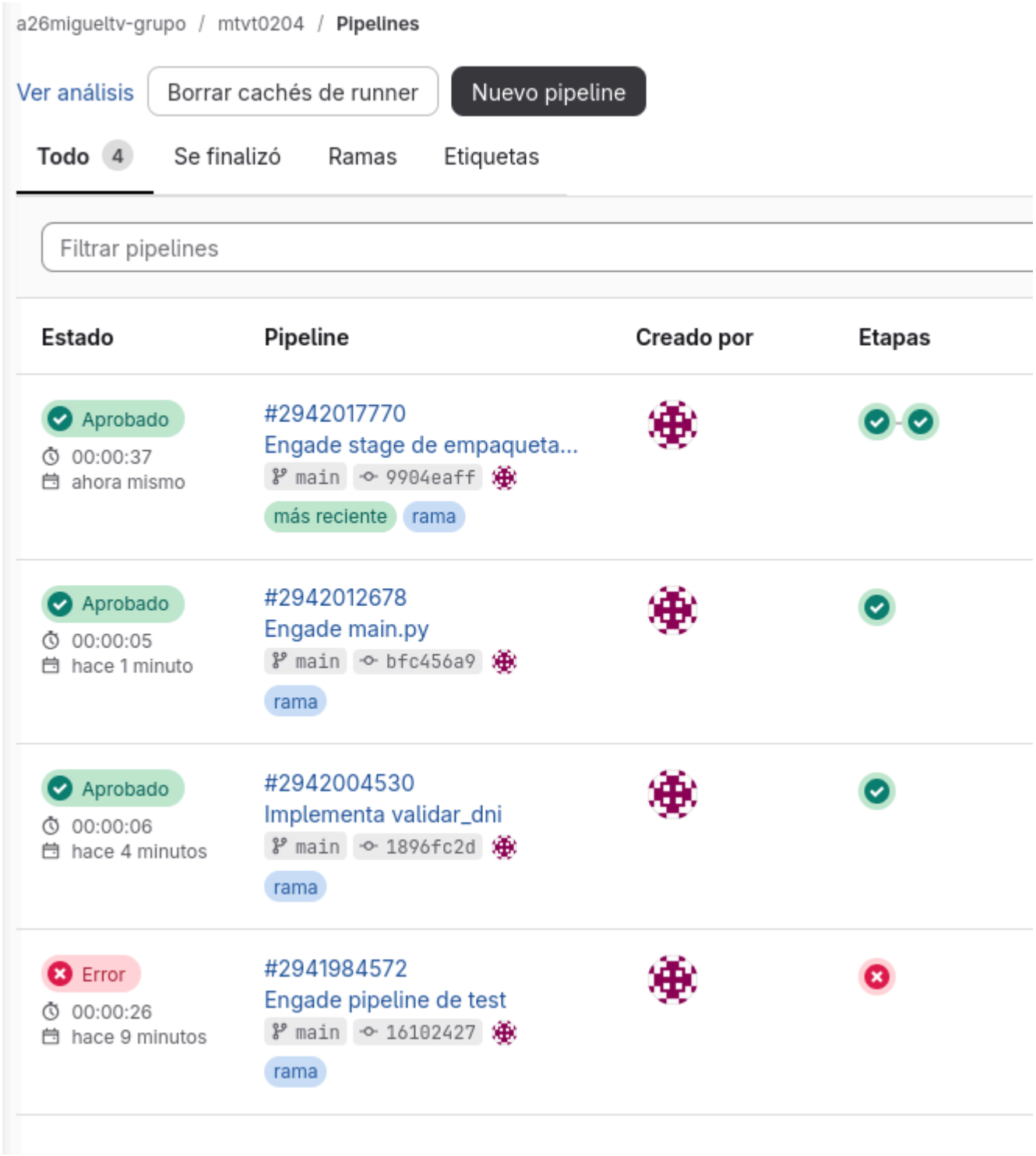
unittest:
  stage: test
  image: python:3.9
  tags:
    - rundock
  script:
    - echo "Comezando os test..."
    - python -m unittest test_dni_validator.py

empaquetar:
  stage: paquetes
  image: python:3.9
  tags:
    - rundock
  script:
    - pip install pyinstaller
    - pyinstaller --onefile main.py
  artifacts:
    paths:
      - dist/main
    expire_in: 1 week
```

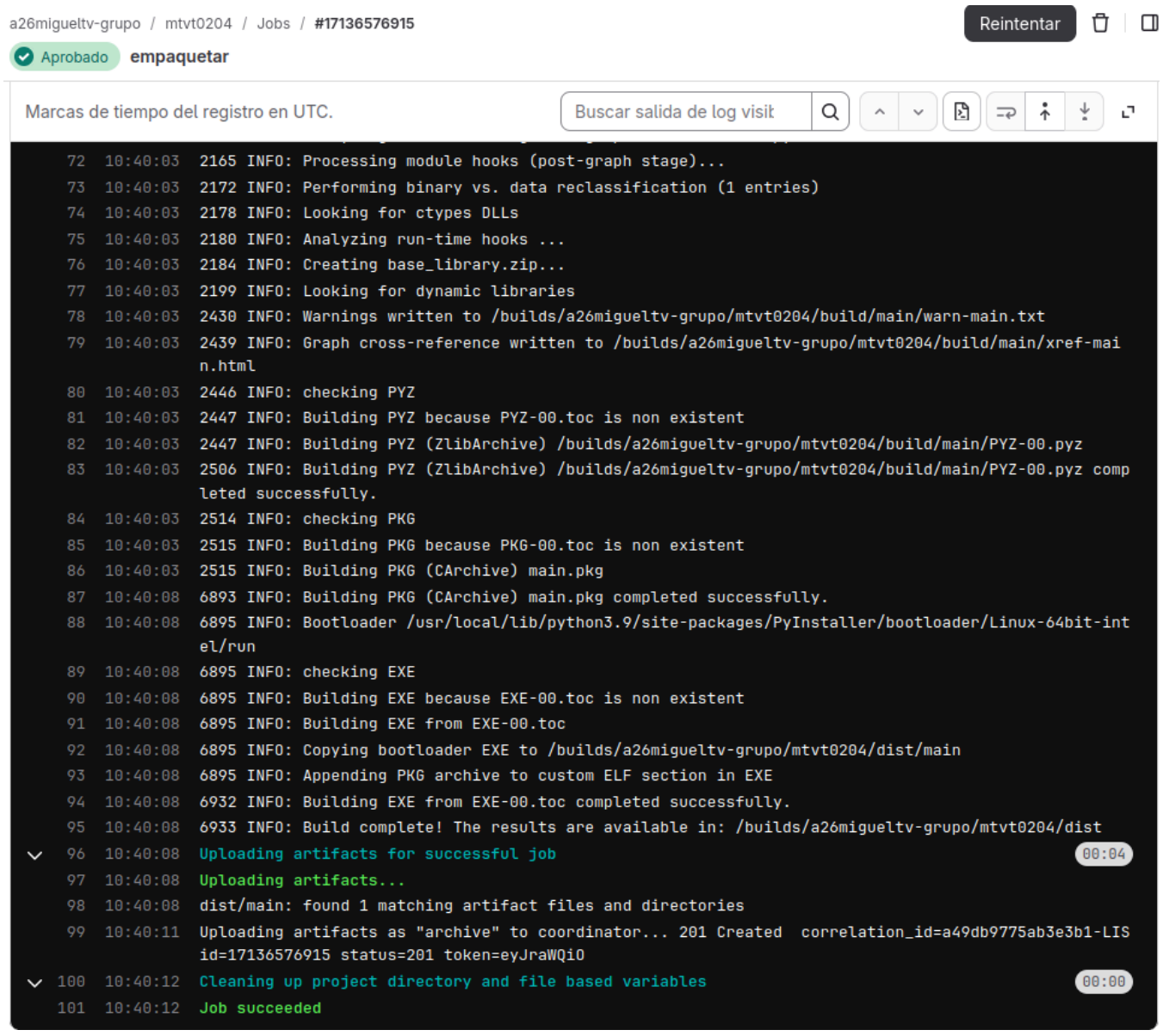
Captura 1 – Contido do ficheiro .gitlab-ci.yml ca nova etapa paquetes e os artefactos



Captura 2 – Build > Pipelines que se executou



Captura 3 – Resumo do job executado de empaquetar

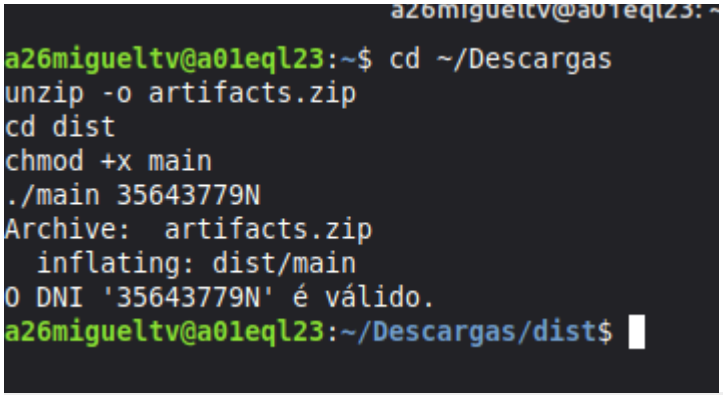


2.3 Descarga e execución do paquete

Dende **Artefactos do job** → **Descargar** baixouse o zip, descomprimiuse e executouse o programa no terminal do equipo:

```
unzip artifacts.zip
cd dist
chmod +x main
./main 35643779N
```

Captura 4 – Execución do paquete xerado



Exercicio 3: Automatización de probas end-to-end

O script só define as funcións de proba pero non as chama, polo que remata sen executar ningunha proba (código de saída 0) e o job aparece como correcto. Para executalas habería que chamalas nun bloque `if __name__ == "__main__":` ou usar `pytest test_main_pexpect.py`.

Engadiuse o ficheiro `test_main_pexpect.py`:

```
import pexpect
import sys
import os

def test_validar_dni_valido():
    dni_valido = "12345678Z"  # Supoñemos que Z é válido
    script_path = os.path.abspath("main.py")

    child = pexpect.spawn(f"{sys.executable} {script_path} {dni_valido}")
    child.expect(f"O DNI '{dni_valido}' é válido.")
    child.expect(pexpect.EOF)  # Espera ao final do proceso

def test_validar_dni_invalido():
    dni_invalido = "12345678A"  # Supoñemos que A é inválido
    script_path = os.path.abspath("main.py")

    child = pexpect.spawn(f"{sys.executable} {script_path} {dni_invalido}")
    child.expect(f"O DNI '{dni_invalido}' NON é válido.")
    child.expect(pexpect.EOF)

def test_sen_argumentos():
    script_path = os.path.abspath("main.py")

    child = pexpect.spawn(f"{sys.executable} {script_path}")
    child.expect("Uso: python main.py <DNI>")
    child.expect(pexpect.EOF)
```

Engadiuse un novo job `e2e` á stage de probas (`test`), que instala `pexpect` e executa a proba no noso runner:

```
stages:
```

```
  - test
  - paquetes

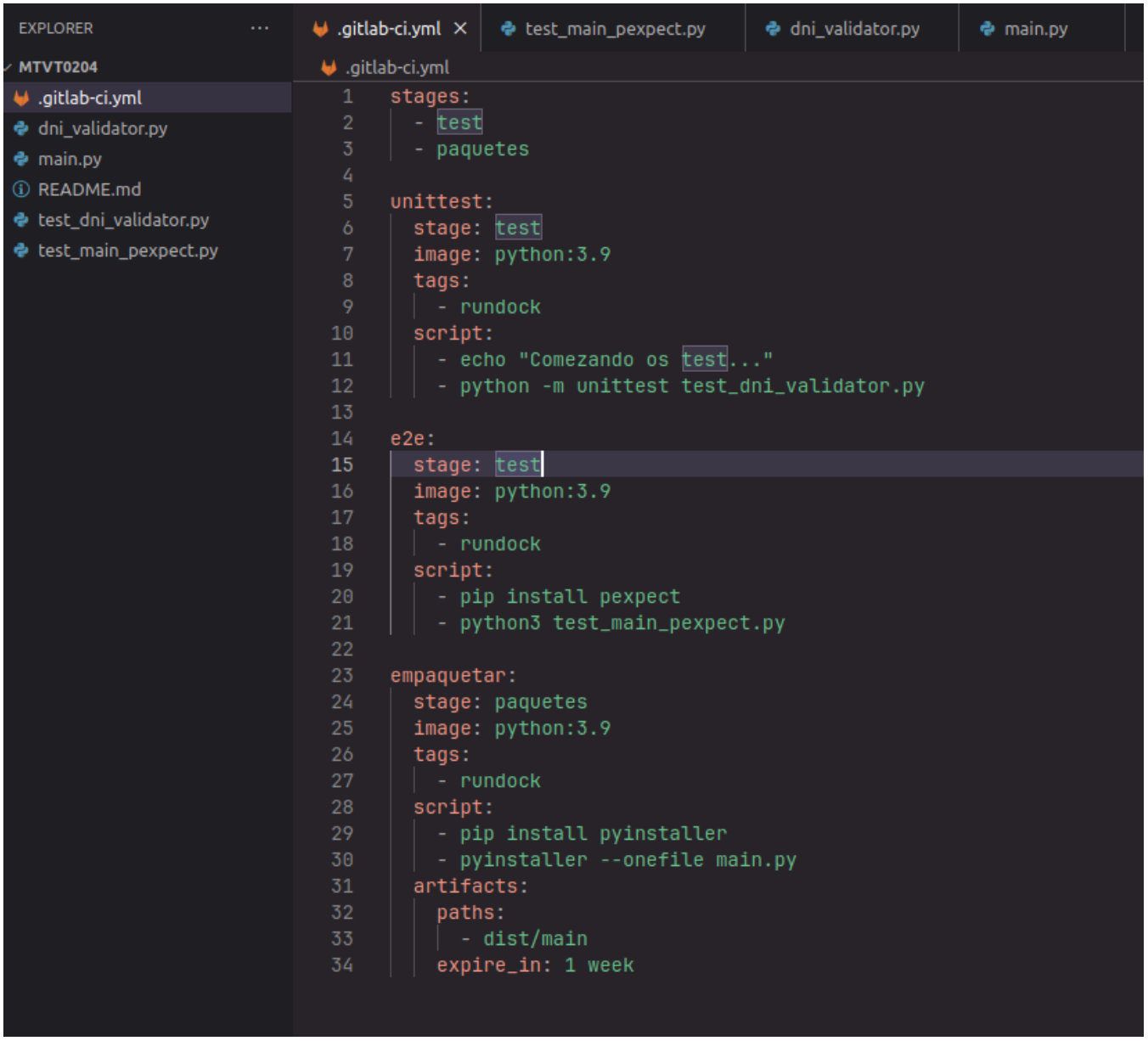
unittest:
  stage: test
  image: python:3.9
  tags:
    - rundock
  script:
    - echo "Comezando os test..."
    - python -m unittest test_dni_validator.py

e2e:
  stage: test
  image: python:3.9
  tags:
    - rundock
  script:
    - pip install pexpect
    - python3 test_main_pexpect.py

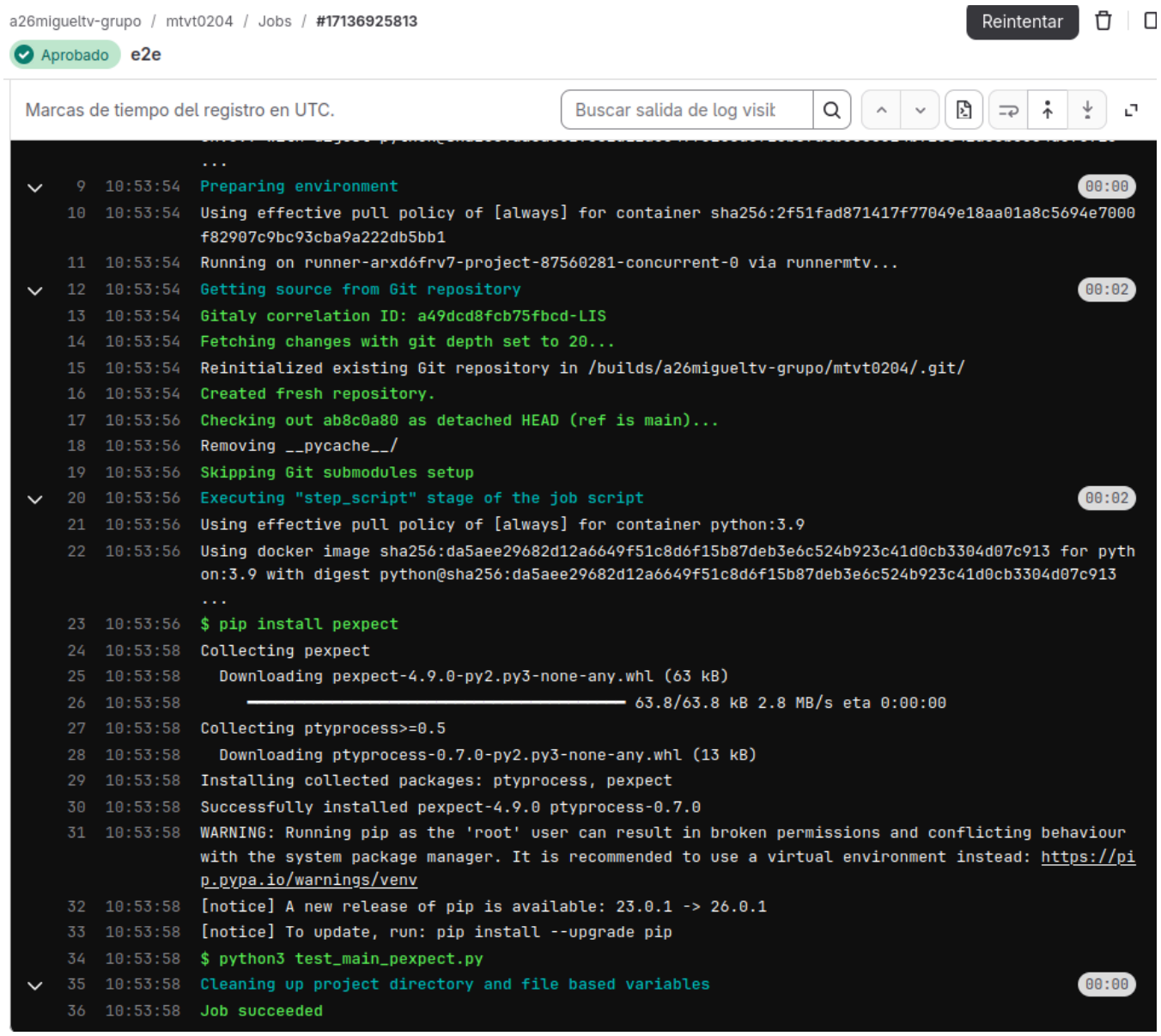
empaquetar:
  stage: paquetes
  image: python:3.9
  tags:
    - rundock
  script:
    - pip install pyinstaller
    - pyinstaller --onefile main.py
  artifacts:
    paths:
      - dist/main
    expire_in: 1 week
```

> Os jobs `unittest` e `e2e` están na mesma stage, polo que se executan en paralelo. A stage `paquetes` só comeza se **os dous** pasan.

Captura 1 – O ficheiro de configuración de CI/CD



Captura 2 – A execución deste job onde se vexan os comandos que se lanzaron na interface web



Captura 3 – O pipeline executado perfectamente na interface web

#2942062238

Aprobado Fecha de creación: hace 1 minuto por **Miguel Tubio vidal**. Fecha de finalización: hace 1 minuto

Para commit `ab8c0a80` Engade proba end-to-end

En `main`

más reciente rama 3 jobs 30 segundos, en cola durante 3 segundos

Pipeline Jobs 3 Pruebas 0

